We follow basically the same procedure as we did for strings; we randomly change a few elements of the parameter vector. The mutation rate parameter controls the number of elements that we change. We need to control the mutation rate carefully to balance the creation of new information that can be used to improve existing solutions and the destruction of old information.

Next we need to assess the fitness of each agent by actually testing them on the environment (CartPole in our case).

Listing 6.10. Testing each agent in the environment

```
import gym
env = gym.make("CartPole-v0")

def test_model(agent):
    done = False
    state = torch.from_numpy(env.reset()).float()
    score = 0
    while not done:
1
        params = unpack_params(agent['params'])
        probs = model(state,params)
2
        action =
torch.distributions.Categorical(probs=probs).sample()  3
        state_, reward, done, info = env.step(action.item())
        state = torch.from_numpy(state_).float()
        score += 1
4
    return score
```

- ***1*** While game is not lost
- ***2*** Gets the action probabilities from the model using the agent's parameter vector
- ***3*** Probabilistically selects an action by sampling from a categorical distribution
- ***4*** Keeps track of the number of time steps the game is not lost as the score

The `test_model` function takes an *agent* (a dictionary of a parameter vector and its fitness value) and runs it in the CartPole environment until it loses the game and returns the number of time steps it lasted as its score. We